🌊 Pacific Atlantic Water Flow / Two Stations Matrix Problem

Complete DSA Notes (File 1A)

Part 1 — Understanding the Problem & Building the Brute Force Solution

Problem Statement

You are given an `n × m` matrix.

Each cell represents the **signal strength** (or height) of a communication tower.

There are **two control stations**.

```
Station P
---------
Top Boundary
Left Boundary

Station Q
---------
Bottom Boundary
Right Boundary
```

A signal can move from one tower to one of its **4 neighbours**

- Up
- Down
- Left
- Right

ONLY IF

```
Neighbour Value <= Current Value
```

This means signals always move from

```
Higher
   ↓
Lower
```

or

```
Equal
```

What is the Question Asking?

This is where many students misunderstand the problem.

The problem is NOT asking

> Find a path.

The problem is NOT asking

> Reach one boundary.

The problem asks

> **How many cells can send a signal to BOTH stations?**

Meaning

For every cell,

ask

```
Can this cell reach Station P?

AND

Can this cell reach Station Q?
```

If both answers are YES,

count that cell.

Understanding Station P and Station Q

Suppose

```
1 2 3
4 5 6
7 8 9
```

Coordinates

```
(0,0) (0,1) (0,2)

(1,0) (1,1) (1,2)

(2,0) (2,1) (2,2)
```

Station P

Station P covers

```
Top Row

AND

Left Column
```

Visual

```
P P P

P . .

P . .
```

These cells can directly transmit to P.

Station Q

Station Q covers

```
Bottom Row

AND

Right Column
```

Visual

```
. . Q

. . Q

Q Q Q
```

These cells can directly transmit to Q.

Signal Movement Rule

This is the MOST IMPORTANT rule.

Signal moves ONLY IF

```
Neighbour <= Current
```

Example

```
5 3
```

Can signal move?

Check

```
3 <= 5

YES
```

So

```
5 ------> 3
```

Another example

```
3 5
```

Check

```
5 <= 3

NO
```

So

```
3   X-----> 5
```

Not allowed.

Think of Water Flow

Imagine every number is height.

```
8
↓
6
↓
4
↓
2
```

Water naturally flows

```
Higher

↓

Lower
```

Exactly the same here.

What are we trying to find?

Suppose this matrix

```
1 2

3 4
```

We examine every cell.

Cell = 1

Question

```
Can 1 reach P?
```

YES

because it is already on P boundary.

Next

```
Can 1 reach Q?
```

No.

So

```
Don't count.
```

Cell = 4

Question

```
Can 4 reach P?
```

Possible path

```
4 → 3
```

3 is on left boundary.

YES.

Now

```
Can 4 reach Q?
```

Already on Q boundary.

YES.

Therefore

```
Count 4.
```

General Thought Process

For every cell

```
Start

↓

Can I reach P?

↓

Can I reach Q?

↓

If both YES

Count++
```

How do we check whether a cell reaches P or Q?

We need to explore every possible path.

Which algorithm explores all possible paths?

```
DFS
```

or

```
BFS
```

DFS is easier here.

Brute Force Thinking

Suppose current cell is

```
(i,j)
```

Start DFS.

Move only if

```
Neighbour <= Current
```

During DFS,

keep checking

```
Did I touch P?

Did I touch Q?
```

If yes,

remember it.

How do we know we reached P?

Very easy.

Current DFS cell

```
(r,c)
```

If

```
r == 0

OR

c == 0
```

then

```
Reached P
```

because

Top row

or

Left column

belongs to P.

How do we know we reached Q?

If

```
r == n-1

OR

c == m-1
```

then

```
Reached Q
```

because

Bottom row

or

Right column

belongs to Q.

Brute Force Algorithm

For every cell

```
Reset

visited

ReachedP

ReachedQ

↓

Run DFS

↓

DFS explores entire reachable region

↓

If reachedP AND reachedQ

answer++
```

Visual Representation

```
Every Cell

      |
      ▼

Create New DFS

      |
      ▼
```

```
Explore

      |
      ▼

Touch Top/Left?

Yes

ReachedP=True

Touch Bottom/Right?

Yes

ReachedQ=True

      |
      ▼

DFS Ends

      |
      ▼

Both True?

Yes

Count++
```

Rough Brute Force Template

```
answer = 0

for every cell:

    visited = new matrix

    reachedP = False

    reachedQ = False

    dfs(i,j)

    if reachedP and reachedQ:

        answer += 1
```

Building DFS

Whenever DFS visits a cell

```
Mark visited
```

Then immediately ask

```
Am I on Top Row?

OR

Am I on Left Column?
```

If yes

```
ReachedP=True
```

Then ask

```
Am I on Bottom Row?

OR

Am I on Right Column?
```

If yes

```
ReachedQ=True
```

Then continue DFS.

DFS Traversal Template

```
Current Cell

↓
```

```
Visit

↓

Check P Boundary

↓

Check Q Boundary

↓

Visit 4 neighbours

↓

Ignore Invalid Cells

↓

Ignore Visited Cells

↓

Check Flow Rule

↓

DFS
```

Brute Force DFS Code

```
dirs = [(-1,0),(1,0),(0,-1),(0,1)]

def dfs(r,c):

    visited[r][c] = True

    if r == 0 or c == 0:
        reachedP[0] = True

    if r == n-1 or c == m-1:
        reachedQ[0] = True

    for dr,dc in dirs:

        nr = r + dr
        nc = c + dc
```

```
        if 0<=nr<n and 0<=nc<m:

            if visited[nr][nc]:
                continue

            if mat[nr][nc] <= mat[r][c]:

                dfs(nr,nc)
```

Complete Brute Force

```
answer = 0

for i in range(n):

    for j in range(m):

        visited = [[False]*m for _ in range(n)]

        reachedP=[False]

        reachedQ=[False]

        dfs(i,j)

        if reachedP[0] and reachedQ[0]:

            answer+=1
```

Dry Run

Suppose

```
1 2

3 4
```

Start from

```
4
```

DFS

```
4

↓

3
```

At 3

```
Left Boundary

ReachedP=True
```

Back to 4

```
Already on Bottom Boundary

ReachedQ=True
```

Final

```
ReachedP=True

ReachedQ=True
```

Answer++

Start from

```
1
```

DFS cannot move anywhere.

Touches

```
Top Boundary
```

ReachedP=True

Never reaches Bottom or Right.

```
ReachedQ=False
```

Don't count.

Time Complexity

Suppose matrix size

```
n × m
```

There are

```
n*m
```

starting cells.

Each DFS can visit

```
n*m
```

cells.

Total

```
O((n*m)*(n*m))

=

O(n²m²)
```

For

```
1000 × 1000
```

this becomes impossible.

The Biggest Problem with Brute Force

Notice something.

Suppose we run DFS from

```
(1,1)
```

Later we run DFS from

```
(1,2)
```

Both DFS traversals explore many of the **same cells again**.

We are repeating the same work hundreds or thousands of times.

This repeated exploration is exactly what leads us to the optimized solution.

Key Observation (Bridge to Optimization)

Instead of repeatedly asking

```
Can THIS cell reach P?
```

ask yourself

```
Can I start from P
and discover ALL cells
that can reach P?
```

This single question is the beginning of the optimized solution.

👉 Part 2 will derive this idea naturally, explain "reverse flow" from first principles, and build the optimized intuition step by step.

Part 2 — Building the Optimized Logic (The Reverse Flow Intuition)

Recap

Brute Force Algorithm

```
For every cell

        ↓

Run DFS

        ↓

Can I reach P?

Can I reach Q?

        ↓

Count
```

Time Complexity

```
O((n*m)*(n*m))
```

The brute force is **correct**.

The only problem is

> **We are repeating the same DFS over and over again.**

Where is the repeated work?

Suppose the matrix is

```
1 2 3

4 5 6

7 8 9
```

Brute force starts

```
DFS(0,0)

↓
```

```
Explore

↓

Many Cells
```

Then

```
DFS(0,1)

↓

Explore

↓

Almost same cells
```

Then

```
DFS(1,1)

↓

Explore

↓

Again same cells
```

Notice something?

We keep visiting

```
5

6

8

9
```

again and again.

The Important Question

Instead of asking

```
Can Cell reach P?
```

Can we ask the opposite?

```
Can P discover every cell
that can reach it?
```

This is the entire optimization.

Why does this even make sense?

Let's understand with one arrow.

Suppose

```
8     6
```

Signal rule

```
Neighbour <= Current
```

Can signal move

```
8 → 6 ?
```

Check

```
6 <= 8

YES
```

So

```
8 -------------> 6
```

Now ask

```
Can 6 send signal to 8?
```

Check

```
8 <= 6

NO
```

So

```
6 X------------> 8
```

Original Direction

Always remember

```
Higher

↓

Lower
```

Example

```
10

↓

8

↓

6
```

```
↓
4
```

Signal moves

```
10
↓
8
↓
6
↓
4
```

Everything is flowing downward.

Now Let's Change the Question

Original Question

```
Can 10 reach P?
```

Brute force

```
10
↓
8
↓
6
↓
P
```

We start from

```
10
```

Instead ask

```
P asks

"Who can reach me?"
```

This is a completely different question.

Imagine P is Alive 😄

Suppose

```
10
↓
8
↓
6
↓
P
```

P asks

```
Who can directly send me a signal?
```

Answer

```
6
```

So P visits

```
6
```

Now P stands on

```
6
```

Again asks

```
Who can send signal to ME?
```

Look carefully.

Original arrow

```
8 -----> 6
```

So who sends signal?

```
8
```

Therefore P moves

```
6 -----> 8
```

Now P stands on

```
8
```

Again asks

```
Who can send signal to 8?
```

Answer

```
10
```

So

```
8 ----->10
```

Finally

P travelled

```
P
↓
6
↓
8
↓
10
```

But notice

Original signal

```
10
↓
8
↓
6
↓
P
```

P walked

```
P
↑
6
↑
8
↑
10
```

It walked in the opposite direction.

THIS is Reverse Flow

We are NOT changing the signal.

Signal still flows

```
10
↓
8
↓
6
↓
P
```

We only changed

who starts the DFS.

Instead of

```
Cell
↓
```

```
P
```

we do

```
P

↓

Cell
```

walking backwards.

We are NOT Reversing Water

Many students think

```
Water now flows upward.
```

❌ Wrong.

Water never changes.

Signal never changes.

Only

```
Search Direction
```

changes.

Real World Analogy

Suppose roads are

```
Home -----> Bus Stop
```

Question

```
Can every home reach bus stop?
```

Brute Force

```
Home1

↓

Bus Stop

Home2

↓

Bus Stop

Home3

↓

Bus Stop
```

Thousands of searches.

Optimized

Stand at

```
Bus Stop
```

Ask

```
Who can reach me?
```

Walk backwards.

One traversal.

Exactly the same idea.

Why Does >= Come?

This is the most memorized line.

Don't memorize.

Let's derive it.

Suppose

Current Reverse DFS Cell

```
6
```

Neighbour

```
8
```

Question

Can I go

```
6 → 8 ?
```

Ask

```
Could 8 send signal to 6
in the ORIGINAL problem?
```

Original Rule

```
Neighbour <= Current
```

If current was

```
8
```

Neighbour

```
6
```

Check

```
6 <= 8

YES
```

Therefore

```
8

↓

6
```

was possible.

Hence during reverse DFS

```
6

↓

8
```

must also be possible.

Mathematical Derivation

Current Reverse Cell

```
6
```

Neighbour

```
8
```

We need

```
Neighbour
```

```
>=

Current
```

Because

```
8 >= 6
```

So reverse DFS rule becomes

```
if neighbour >= current:
```

NOT

```
<=
```

Reverse DFS Rule

Original DFS

```
Current

↓

Neighbour

if

Neighbour <= Current
```

Reverse DFS

```
Current

↓

Neighbour

if
```

```
Neighbour >= Current
```

Only one sign changes.

Everything else stays the same.

New Thinking

Brute Force

```
Cell1

↓

P ?
```

```
Cell2

↓

P ?
```

```
Cell3

↓

P ?
```

Thousands of DFS.

Optimized

```
P

↓

Find ALL Cells
that can reach P.
```

One DFS.

But P Has Many Cells!

Excellent observation.

P is not one cell.

P is

```
Entire Top Row

+

Entire Left Column
```

Example

```
P P P P

P . . .

P . . .

P . . .
```

Every one of these cells belongs to P.

So start DFS from

```
Every P Boundary Cell
```

Same for Q

```
. . . Q

. . . Q

. . . Q

Q Q Q Q
```

Start DFS from

```
Every Q Boundary Cell
```

Why Doesn't This Become Slow?

Question

```
Top Row has m cells

Left Column has n cells

Won't we run many DFS?
```

Yes.

But remember

```
Visited Matrix
```

Suppose DFS from

```
(0,0)
```

already visited

```
(1,0)

(2,0)

(2,1)
```

Later

DFS starts from

```
(0,1)
```

Those cells are already

```
Visited
```

So DFS immediately returns.

Every cell is visited

```
Only Once
```

for Pacific

and

```
Only Once
```

for Atlantic.

Final Algorithm Idea

Create

```
Pacific Matrix

Atlantic Matrix
```

Pacific DFS

```
Start

Top Row

+

Left Column
```

Mark

```
Every reachable cell
```

Atlantic DFS

```
Start

Bottom Row

+

Right Column
```

Mark

```
Every reachable cell
```

Finally

```
For every cell

if

Pacific=True

AND

Atlantic=True

Count++
```

Visualization

Pacific Search

```
Top + Left

↓↓↓↓↓↓↓

Mark every reachable cell.
```

Atlantic Search

```
Bottom + Right

↑↑↑↑↑↑↑↑

Mark every reachable cell.
```

Intersection

```
Pacific

AND

Atlantic
```

Answer.

The Biggest Intuition

Brute Force asks

```
Can THIS cell reach P?
```

Optimized asks

```
Starting from P,

which cells are capable
of reaching me?
```

Both questions have

the same answer.

But

one requires

```
n*m DFS
```

the other requires only

```
2 DFS traversals
```

Mental Model (Remember Forever)

Whenever you see

```
Can every node
reach Destination?
```

Don't immediately start DFS from every node.

Instead ask

```
Can I start from Destination

and walk backwards?
```

This one question has appeared in

- Pacific Atlantic Water Flow
- Reverse Graph Problems
- Multi Source BFS
- Many Grid DFS Interview Problems

Transition to File 1C

Now we know

- Why brute force is slow ✅
- Why reverse search works ✅
- Why >= appears ✅
- Why two visited arrays are enough ✅

The only thing left is implementation.

Next Part will build the optimized code line by line, perform a complete dry run, explain every line, discuss edge cases, complexity, and common interview mistakes.

Part 3 — Building the Optimized Code Step by Step

Recap

After optimization we know

```
Step 1

Find all cells
that can reach Pacific.

↓

Step 2

Find all cells
that can reach Atlantic.

↓

Step 3

Intersection.
```

Now we only need to implement this idea.

Step 1 — Create Two Visited Matrices

Question

What information do we need?

We need to remember

```
Can Reach Pacific?

Can Reach Atlantic?
```

So create

```
n = len(mat)
m = len(mat[0])

pacific = [[False]*m for _ in range(n)]
atlantic = [[False]*m for _ in range(n)]
```

Meaning

```
pacific[i][j] = True
```

means

```
Cell (i,j)

CAN reach Pacific
```

Similarly

```
atlantic[i][j] = True
```

means

```
Cell (i,j)

CAN reach Atlantic
```

Notice

We are NOT storing paths.

Only

```
Reachable

or

Not Reachable
```

Step 2 — Direction Array

Normal DFS.

```
dirs = [
    (-1,0),
    (1,0),
```

```
    (0,-1),
    (0,1)
]
```

Nothing new.

Step 3 — Build DFS

Question

What should DFS receive?

We need

```
Current Row

Current Column

Visited Matrix
```

because

sometimes DFS is for

```
Pacific
```

sometimes

```
Atlantic
```

So

```
def dfs(r,c,visited):
```

Step 4 — Mark Current Cell

Whenever DFS reaches a cell

Mark it.

```
visited[r][c]=True
```

Meaning

```
This cell can reach
this ocean.
```

Step 5 — Explore Four Directions

Standard DFS.

```
for dr,dc in dirs:

    nr=r+dr
    nc=c+dc
```

Step 6 — Boundary Check

```
if 0<=nr<n and 0<=nc<m:
```

Ignore invalid cells.

Step 7 — Skip Visited Cells

```
if visited[nr][nc]:
    continue
```

Otherwise

DFS may become infinite.

Step 8 — Reverse Flow Condition

This is the ONLY important line.

Original

```
Neighbour <= Current
```

Reverse

```
Neighbour >= Current
```

Therefore

```
if mat[nr][nc] >= mat[r][c]:

    dfs(nr,nc,visited)
```

Remember

We are asking

```
Could neighbour
have flowed into me?
```

Not

```
Can I flow into neighbour?
```

Complete DFS

```
def dfs(r,c,visited):

    visited[r][c]=True

    for dr,dc in dirs:

        nr=r+dr
        nc=c+dc

        if 0<=nr<n and 0<=nc<m:
```

```
            if visited[nr][nc]:
                continue

            if mat[nr][nc] >= mat[r][c]:

                dfs(nr,nc,visited)
```

Notice

Compared to brute force

Only ONE line changed.

```
<=

became

>=
```

Everything else is normal DFS.

Step 9 — Pacific DFS

Question

Where should Pacific DFS start?

Pacific covers

```
Top Row

+

Left Column
```

Example

```
P P P P

P . . .

P . . .
```

```
P . . .
```

Therefore

Top Row

```
for j in range(m):
    if not pacific[0][j]:
        dfs(0,j,pacific)
```

Left Column

```
for i in range(n):
    if not pacific[i][0]:
        dfs(i,0,pacific)
```

Now

```
Pacific Matrix
is completely filled.
```

Step 10 — Atlantic DFS

Atlantic covers

```
Bottom Row
+
Right Column
```

Bottom Row

```
for j in range(m):
    if not atlantic[n-1][j]:
        dfs(n-1,j,atlantic)
```

Right Column

```
for i in range(n):
    if not atlantic[i][m-1]:
        dfs(i,m-1,atlantic)
```

Done.

Step 11 — Count Intersection

Now

Pacific Matrix

looks something like

```
T T T F
T T T F
T T F F
F F F F
```

Atlantic

```
F F T T
F T T T
F T T T
T T T T
```

Now

Loop through every cell.

```
answer=0

for i in range(n):

    for j in range(m):

        if pacific[i][j] and atlantic[i][j]:

            answer+=1
```

Return

```
return answer
```

Done.

Complete Optimized Code

```
def countCells(mat):

    n=len(mat)
    m=len(mat[0])

    pacific=[[False]*m for _ in range(n)]
    atlantic=[[False]*m for _ in range(n)]

    dirs=[(-1,0),(1,0),(0,-1),(0,1)]

    def dfs(r,c,visited):

        visited[r][c]=True

        for dr,dc in dirs:

            nr=r+dr
            nc=c+dc

            if 0<=nr<n and 0<=nc<m:

                if visited[nr][nc]:
                    continue

                if mat[nr][nc] >= mat[r][c]:
```

```
                    dfs(nr,nc,visited)

        # Pacific
        for j in range(m):
            if not pacific[0][j]:
                dfs(0,j,pacific)
        for i in range(n):
            if not pacific[i][0]:
                dfs(i,0,pacific)
        # Atlantic
        for j in range(m):
            if not atlantic[n-1][j]:
                dfs(n-1,j,atlantic)
        for i in range(n):
            if not atlantic[i][m-1]:
                dfs(i,m-1,atlantic)
        answer=0
        for i in range(n):
            for j in range(m):
                if pacific[i][j] and atlantic[i][j]:
                    answer+=1
        return answer
```

Dry Run

Matrix

```
1 2
```

```
3 4
```

Pacific DFS Starts

Starts from

```
1

2

3
```

From

```
1
```

Can go to

```
2

because

2>=1
```

Can also go to

```
3

because

3>=1
```

From

```
2
```

Can go to

```
4
```

```
because

4>=2
```

Eventually

Pacific becomes

```
T T

T T
```

Atlantic DFS Starts

Starts from

```
4

3

2
```

Again

every cell becomes reachable.

Atlantic

```
T T

T T
```

Intersection

```
T T

T T
```

Answer

```
4
```

Why Is Time Complexity O(n*m)?

Students usually think

```
Top Row

m DFS

+

Left Column

n DFS
```

Will become large.

Actually

Visited Matrix saves us.

Suppose

```
DFS(0,0)

already visited

(1,0)

(2,0)

(2,1)
```

Later

```
DFS(0,1)
```

immediately returns

because

```
Visited=True
```

Every cell is visited

```
At Most Once
```

during Pacific DFS.

Again

```
At Most Once
```

during Atlantic DFS.

Therefore

```
O(n*m)
```

Complexity

Time

```
Pacific DFS

O(n*m)

+

Atlantic DFS

O(n*m)

+

Counting

O(n*m)
```

Overall

```
O(n*m)
```

Space

```
Pacific Matrix

O(n*m)

Atlantic Matrix

O(n*m)

DFS Stack

O(n*m)
```

Common Mistakes

Mistake 1

Using

```
<=
```

instead of

```
>=
```

Remember

Reverse Search.

Mistake 2

Starting DFS from every cell.

That is brute force.

Mistake 3

Creating new visited matrix inside every DFS.

Wrong.

Need only

```
One Pacific Matrix

One Atlantic Matrix
```

Mistake 4

Forgetting

```
Left Column

Top Row

Bottom Row

Right Column
```

Need all four.

Mistake 5

Thinking

```
We reversed water.
```

Wrong.

Water never changes.

Only

```
DFS Direction
```

changes.

Final Mental Picture

Brute Force

```
Every Cell

↓

Pacific?
```

```
Every Cell

↓

Atlantic?
```

Millions of searches.

Optimized

```
Pacific

↓

Find ALL Cells
```

```
Atlantic

↓

Find ALL Cells
```

```
Intersection
```

Done.

Final Interview Summary

If interviewer asks

"How did you think?"

Say

> "The brute-force solution runs DFS from every cell and repeatedly explores the same regions. Instead, I reversed the search direction. Rather than asking whether each cell can reach an ocean, I started DFS from the ocean boundaries and traversed in reverse (`neighbor >= current`). This marks every cell that can reach that ocean. Running this once for Pacific and once for Atlantic, then taking the intersection of the two visited matrices, gives the answer in `O(n*m)`."

End of Learning Notes

You now know

✅ Problem Understanding

✅ Brute Force

✅ Why Brute Force Fails

✅ Reverse Flow Intuition

✅ Reverse Graph Thinking

✅ Multi-Source DFS

✅ Code Building

✅ Dry Run

✅ Complexity

🌊 Pacific Atlantic Water Flow / Two Stations Matrix Problem

PART 4 — Permanent DSA Pattern Sheet (Interview Revision)

📌 Pattern Name

```
Reverse Flow + Multi-Source DFS/BFS
```

Also Known As

```
Reverse Graph Traversal

Multi-Source DFS

Reverse Reachability

Ocean Flow Pattern
```

The Core Idea

Instead of asking

```
Can every node
reach Destination?
```

Ask

```
Can Destination
discover every node
that can reach it?
```

This single question converts

```
O((n*m)^2)

↓

O(n*m)
```

The Pattern

Suppose interview asks

```
Find all cells
that can reach X.
```

Most beginners think

```
Every Cell

↓

DFS

↓

Reach X?
```

Wrong direction.

Instead think

```
X

↓

Reverse DFS

↓

Find every reachable cell.
```

Recognition Pattern

Whenever you read

```
Can reach...
```

Immediately ask yourself

```
Can I reverse the search?
```

Recognition Keywords

These words should trigger this pattern.

```
Reach

Flow

Spread

Propagate

Transmit

Travel

Escape

Drain

Water

Signal

Ocean

Boundary

Exit

Border

Destination

Can reach

Eventually reach

Connected to
```

Recognition Checklist

While reading the problem ask

```
1.

Am I checking
whether every node
reaches one destination?

YES
```

```
↓

Go Next.
```

```
2.

Will DFS from every node
repeat work?

YES

↓

Go Next.
```

```
3.

Can I start
from the destination?

YES

↓

Go Next.
```

```
4.

Can I reverse
the movement rule?

YES

↓

Reverse DFS/BFS.
```

Biggest Observation

Brute Force

```
Cell

↓

Destination?
```

Optimization

```
Destination

↓

Cell
```

Nothing else changes.

Reverse Flow Rule

Original

```
Neighbour <= Current
```

Reverse

```
Neighbour >= Current
```

General Formula

```
Reverse

=

Reverse every edge.
```

Important Interview Question

Interviewer

```
Why >= ?
```

Answer

```
Because during reverse DFS
I'm asking

"Could this neighbour
have reached me
in the original graph?"

If yes,

I move there.
```

Never say

```
Because reverse flow.
```

Explain WHY.

Interview Thinking Framework

Always think in this order.

Step 1

Understand movement.

```
How can I move?
```

Step 2

Find destination.

```
Where am I trying to reach?
```

Step 3

Brute Force

```
Run DFS

from every node.
```

Step 4

Observe repetition.

```
Same cells

visited

again and again.
```

Step 5

Reverse the question.

Instead of

```
Can node reach destination?
```

Think

```
Can destination
find all nodes?
```

Step 6

Reverse movement.

```
<=

↓

>=
```

Step 7

Run Multi-Source DFS.

Step 8

Collect answer.

Generic Algorithm Template

```
Destination Cells

↓

Push into DFS/BFS

↓

Reverse Movement

↓

Mark Reachable

↓

Repeat for every destination

↓

Intersection

↓

Answer
```

Universal Code Template

```
visited=[[False]*m for _ in range(n)]

def dfs(r,c):

    visited[r][c]=True

    for neighbour:

        if valid:

            if visited:

                continue

            if reverse_condition:

                dfs(neighbour)
```

Then

```
for every source:

    dfs(source)
```

Done.

How to Identify Reverse Graph Problems

If interviewer asks

```
Can every node
reach one node?
```

Think

```
Reverse Graph.
```

If interviewer asks

```
Can every city
reach capital?
```

Think

```
Start from capital.
```

If interviewer asks

```
Can every room
reach exit?
```

Think

```
Start from exit.
```

If interviewer asks

```
Can every student
reach principal office?
```

Think

```
Start from office.
```

Real World Analogies

Example 1

Bus Stop

Question

```
Which houses
can reach Bus Stop?
```

Wrong

```
House1

↓

Bus Stop
```

```
House2

↓

Bus Stop
```

Correct

```
Bus Stop

↓

Walk backwards

↓

Find every house.
```

Example 2

River

Question

```
Which villages
can send water
to river?
```

Wrong

```
Village

↓

River
```

Correct

```
River

↓

Move uphill

↓

Find villages.
```

Example 3

Internet

Question

```
Which computers
can send packets
to server?
```

Wrong

```
Computer

↓

Server
```

Correct

```
Server

↓

Reverse connections
```

```
↓

Find computers.
```

Why Multi-Source DFS?

Pacific is NOT

```
One Cell.
```

Pacific is

```
Entire Boundary.
```

Therefore

Sources

```
Top Row

+

Left Column
```

Similarly

Atlantic

```
Bottom Row

+

Right Column
```

Whenever destination has

```
Many starting points

↓
```

```
Multi-Source DFS/BFS
```

should click.

Common Mistakes

Mistake 1

Memorizing

```
>=
```

Instead derive it.

Mistake 2

Thinking

```
Water reversed.
```

Wrong.

Search reversed.

Mistake 3

Starting DFS

from every cell.

Mistake 4

Forgetting

Visited Matrix.

Mistake 5

Running DFS

without reverse condition.

Similar Interview Problems

1. Pacific Atlantic Water Flow (LeetCode 417)

Exactly this pattern.

---

2. Surrounded Regions (LeetCode 130)

```
Border

↓

Reverse DFS

↓

Mark Safe Cells
```

Pattern

```
Reverse Boundary DFS
```

---

3. Number of Enclaves (LeetCode 1020)

```
Boundary

↓

Remove reachable land

↓

Remaining answer.
```

Same idea.

---

4. Walls and Gates (LeetCode 286)

```
All Gates
```

```
↓

Multi-Source BFS

↓

Fill Distances.
```

---

5. Rotting Oranges (LeetCode 994)

```
All Rotten

↓

Multi-Source BFS.
```

---

6. 01 Matrix (LeetCode 542)

```
All Zeroes

↓

Multi-Source BFS.
```

---

7. Escape Large Maze

Reverse thinking appears.

---

8. Reverse Graph Reachability

Any graph problem asking

```
Who can reach destination?
```

Pattern Family

```
Grid DFS
```

```
↓

Boundary DFS

↓

Reverse DFS

↓

Multi Source DFS

↓

Reverse Graph
```

These are all closely related.

Complexity Pattern

Brute Force

```
Nodes

×

DFS

=

O(V²)
```

Reverse DFS

```
One Traversal

=

O(V)
```

Grid

```
O(n*m)
```

Questions to Ask Yourself During Interview

1.

Am I starting DFS
from every node?

2.

Am I repeating work?

3.

Can I reverse
the search?

4.

Can destination
be my starting point?

5.

Do I have
multiple destinations?

↓

Multi-Source DFS?

6.

What happens

```
to movement rule
after reversing?
```

One-Line Recognition Rule

```
Node → Destination ?

↓

Think

Destination → Node
```

30-Second Revision Sheet

```
Problem

↓

Can node reach destination?

↓

Brute Force

DFS from every node

↓

Repeated work

↓

Reverse Search

↓

Start from destination

↓

Reverse movement

↓
```

```
Multi-Source DFS
↓
Visited Matrix
↓
Intersection
↓
Answer
```

10-Second Interview Cheat Sheet

```
Destination?
↓
Reverse DFS
↓
>=
↓
Multi Source
↓
Visited
↓
Intersection
```

Master Formula

Whenever you see

```
Can every node
reach X?
```

Immediately think

```
Start from X

↓

Reverse DFS/BFS

↓

Mark all reachable nodes

↓

Done.
```

Final Takeaway

This problem is **not about matrices**.

It is **not about water**.

It is **not about DFS**.

The real interview pattern is:

> **Whenever many nodes independently ask "Can I reach the same destination?", consider reversing the search. Start from the destination (or all destination boundary cells), traverse the graph in reverse, and mark every node that can reach it.**

Once this pattern clicks, you'll recognize it in many graph and grid problems—not just Pacific Atlantic Water Flow.